

SECURITY MONITORING SYSTEM ON AWS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Anil kumar, APSSDC

Submitted by

***Kalagara Kusuma (23MH1A0430),
Aketi Lakshmi Srujana (23MH1A4902) ,
Chevvakula Sireesha (23MH1A4911),
Addepalli Ramya Hari Priya Varshini (23MH1A4951)
Rayudu Satya Sri(23MH1A4953)***

ADITYA COLLEGE OF ENGINEERING AND TECHNOLOGY

(July),2025

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for us, and we are truly thankful for the support, guidance, and encouragement we received along the way.

First and foremost, We extend our sincere regards to Mrs. Sumana Bethala and Mr. Anil Kumar Varanasi, our supervisors and mentors, for providing us with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

We would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback we received from my colleagues significantly contributed to the development and success of this project.

In conclusion, we are honoured to have been a part of this internship program, and we look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Kalagara Kusuma(23MH1A0430) - kalagarakusuma2005@gmail.com

Aketi Lakshmi Srujana (23MH1A4902)-

alakshmisrujana239@gmail.com

Chevvakula Sireesha(23MH1A4911)- sireesha5552@gmail.com

Addepalli Ramya Hari Priya

Varshini(23MH1A4951)priyavarshini42005@gmail.com

Rayudu Satya Sri(23MH1A4953)-

Satyasrirayudul@gmail.com

ABSTRACT

This document outlines the importance and architecture of a robust security monitoring system built on Amazon Web Services (AWS). The increasing complexity of cloud environments and the shared responsibility model necessitates effective monitoring to protect applications, data, and infrastructure against evolving threats.

- **Data Collection & Logging:** Leveraging AWS services like CloudTrail for API activity auditing, VPC Flow Logs for network traffic monitoring, and CloudWatch for logs and custom metrics collection.
- **Centralized Security Hub:** Utilizing AWS Security Hub to aggregate security findings from various AWS services and partner solutions, providing a consolidated view of the security posture.
- **Threat Detection & Analysis:** Implementing services like Amazon GuardDuty for continuous threat detection using machine learning and anomaly detection, and potentially leveraging tools like Amazon Inspector for vulnerability assessments.
- **Alerting & Notification:** Configuring CloudWatch alarms and Amazon SNS to provide timely notifications of security events, allowing for prompt response and remediation.
- **Automated Response & Remediation:** Implementing AWS Lambda functions to automate actions based on security alerts, such as isolating compromised resources or triggering remediation workflows.
- **Identity and Access Management (IAM):** Implementing strong IAM policies and roles to enforce the principle of least privilege. AWS IAM, along with services like AWS Organizations and AWS Single Sign-on (SSO), helps control who can access what, and ensures secure authentication and authorization.
- **Encryption & Data Protection:** Enabling encryption for data at rest and in transit using AWS Key Management Service (KMS), SSL/TLS, and S3 bucket policies to ensure sensitive data is protected at all times.
- **Compliance & Audit Readiness:** Using AWS Config and AWS Audit Manager to track resource configurations and ensure continuous compliance with organizational and industry-specific standards (e.g., ISO, HIPAA, GDPR).

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	Introduction.....	5
2.	Methodology.....	7
3.	System Design / Architecture.....	9
4.	Implementation.....	16
5.	Results	19
6.	Conclusion.....	23

1.INTRODUCTION

- Ever wondered how we can keep tabs on who's accessing our most sensitive data in AWS? Whether it's API keys, database credentials, or other critical secrets, every access poses a potential security risk. That's why organizations invest heavily in robust monitoring systems to track and alert on any unusual activity.

- In this group project, we'll build our own powerful monitoring system using AWS. Protecting data is an essential skill for roles like Security Engineer, DevOps Engineer, and Systems or Cloud Administrator, and by working together we'll gain hands-on experience in designing, deploying, and maintaining an end-to-end audit and alerting solution.

Building a security monitoring system on AWS is a critical aspect of maintaining a secure and reliable cloud environment. This comprehensive system involves continuously observing and analyzing various AWS cloud resources, applications, and services to proactively detect, diagnose, and address security threats and potential breaches.

In this project, we combined several AWS services—**CloudTrail**, **CloudWatch**, **Secrets Manager**, and **SNS**—to create a practical and reliable alerting system. By simulating access to sensitive data and configuring alarms to respond in real time, we learned how crucial **observability**, **event tracking**, and **notification pipelines** are in cloud security. Beyond the technical implementation, the project fostered critical thinking around system design, proper threshold settings, and error diagnostics. It demonstrated how a misconfigured metric or a missing subscription confirmation could silently break an otherwise functional system. Most importantly, this experience emphasized the real-world importance of integrating detection and response into every security workflow.

❖ **Centralized Logging and Monitoring:**

- CloudTrail:

This service records API calls made on your AWS account, providing an audit trail of all actions performed. This is crucial for tracking user activity and identifying potential security breaches.

- CloudWatch:

A versatile monitoring service that collects metrics and logs from various AWS services and your applications. CloudWatch enables you to set up alarms based on specific thresholds, triggering notifications when predefined conditions are met.

- AWS Config:

This service enables you to assess, audit, and evaluate the configuration of your AWS resources. It helps ensure compliance with security best practices and identifies misconfigurations that could lead to vulnerabilities.

- Security Hub:

This service aggregates security alerts from multiple AWS services and third-party solutions, providing a unified view of your security posture. It prioritizes alerts based on severity, helping security teams focus on the most critical issues.

2.METHODOLOGY

Building a security monitoring system on AWS follows a phased approach—starting with foundational services like **CloudTrail** and **CloudWatch** for logging and basic monitoring, then progressing to advanced features such as **SIEM integration**, **automated remediation**, and **threat intelligence**. Leveraging AWS native services and integrating third-party tools ensures a comprehensive solution. This approach not only enhances visibility but also strengthens an organization's overall security posture.

Requirements Gathering and Planning

Begin by defining your security monitoring goals. Identify the data sources, compliance requirements, and the organization's overall security posture. This step ensures the system is tailored to specific needs.

AWS Service Selection and Configuration

Next, select and configure AWS services such as **CloudWatch**, **CloudTrail**, **GuardDuty**, and **Security Hub**. Set up data collection, IAM roles, and permissions to ensure secure access and visibility.

Data Collection and Ingestion

Collect logs and metrics from AWS resources and ingest them into **CloudWatch** and **Security Hub**. Define data retention and storage policies to meet compliance and operational needs.

Data Processing and Analysis

Process and analyze the collected data to detect threats. Use **CloudWatch Logs Insights** for querying logs and **Security Hub** for centralized findings. Create custom detection rules as needed to surface specific threats. Timely analysis and alerting help security teams respond quickly to potential incidents.

❑ Alerting and Notification:

Once the security threats and incidents are identified, alerts and notifications need to be sent to the security team. CloudWatch Alarms can be set up for security events, and SNS can be configured for notifications. Integration with incident response tools can also be done to automate the incident response process.

❑ Incident Response and Remediation:

A well-defined incident response plan is critical to respond to security incidents effectively. The incident response plan should include procedures for containment, eradication, recovery, and post-incident activities. Automation can be used to respond to security incidents using Lambda or Step Functions.

❑ Continuous Monitoring and Improvement:

Finally, the security monitoring system needs to be continuously monitored and improved. This involves monitoring system performance and security posture, updating configurations and rules as needed, and conducting regular security assessments and audits. Continuous monitoring ensures that the security monitoring system remains effective and efficient in identifying and responding to security threats.

3. SYSTEM DESIGN ARCHITECTURE

To maintain a high standard of security, visibility, and compliance within cloud-based applications such as the **Student Portfolio Management System**, AWS provides a scalable and automated **Security Monitoring System**. This system is architected to detect unauthorized access to sensitive resources, specifically focusing on **Secrets Manager**, and to notify administrators in real-time for immediate response.

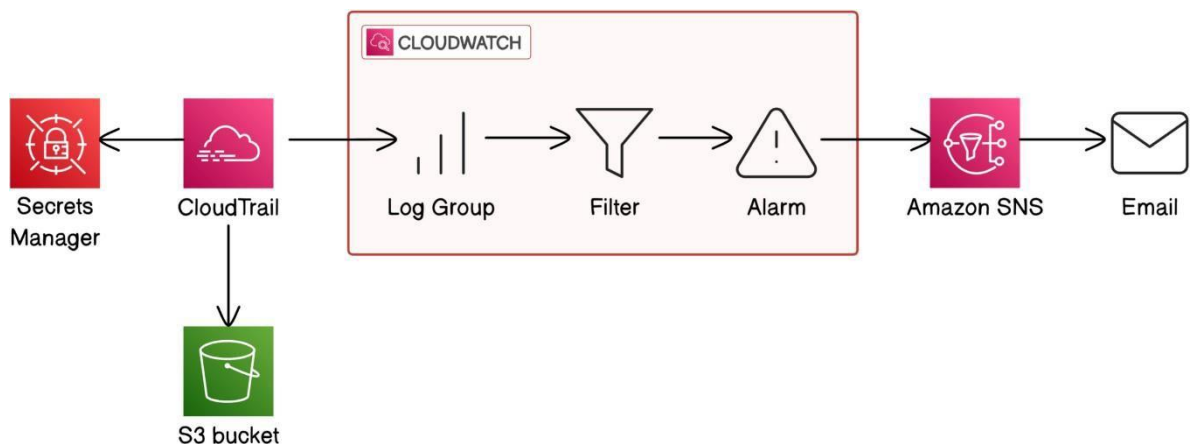
3.1. Secrets Manager Function:

Securely manages sensitive information such as API keys and passwords.

Security Feature:

Automates rotation, enforces fine-grained access control, and provides secure storage.

Figure 3.1: Architecture Diagram



3.2. CloudTrail Function:

Enables governance, compliance, and operational auditing of AWS accounts.

Logging: Records API calls made on your account, capturing actions taken by users, roles, or AWS services.

3.3. S3 Bucket Function:

Stores logs generated by CloudTrail, ensuring durable, secure, and scalable storage solutions.

Access: Facilitates further analysis and compliance audits by retaining records of AWS account activity.

4. CloudWatch Function:

Provides monitoring and observability of AWS resources and applications.

Log Group: Collects logs from various AWS services, enabling metric creation and alarm setup based on incoming log data.

5. Filter Purpose:

Analyzes log data for specific patterns or conditions, helping to identify noteworthy events or anomalies.

Configuration :Can be customized to detect specific log patterns indicative of security breaches or performance issues.

6. Alarm Function:

Initiates notifications when predefined thresholds in CloudWatch are met, ensuring timely responses to critical events.

Integration: Triggers designated actions based on specified conditions.

7. Amazon SNS (Simple Notification Service) Function:

Facilitates the distribution of alerts and announcements to subscribers.

Communication:

Supports multiple protocols, including email, SMS, and application endpoints.

8. Email Notification Purpose:

Provides immediate alerts to designated personnel regarding critical events.

Integration:

Enables quick responses to system alerts or warnings through automated email notifications.

Architectural Flow

1. Secrets Management:

Secrets are securely stored and managed by the Secrets Manager.

2. API Call Logging:

CloudTrail logs all API calls and stores the logs in an S3 bucket for retention and subsequent analysis.

3. Monitoring:

CloudWatch monitors log data, aggregating logs into groups for further processing.

Logs are filtered based on predefined criteria to identify any issues or events requiring attention.

4. Analysis:

Logs are filtered based on predefined criteria to identify any issues or events requiring attention.

5. Alerting:

When certain thresholds are met, alarms in CloudWatch trigger alerts via Amazon SNS.

6. Notification:

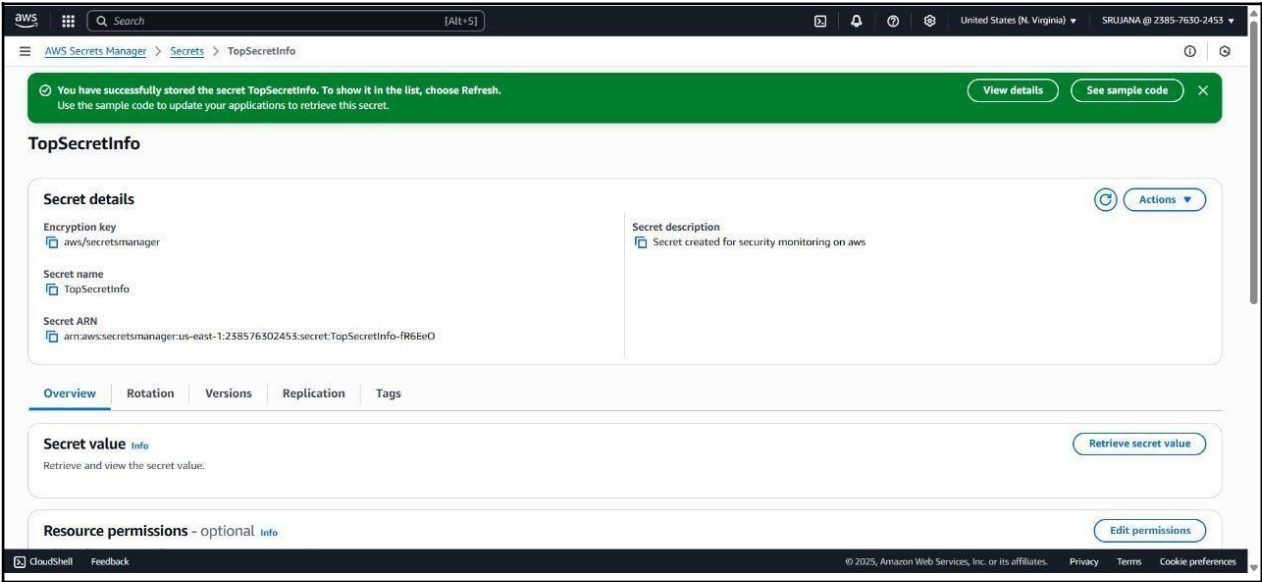
Alerts are disseminated to designated recipients via email, providing real-time information about the triggered alarm.

4. IMPLEMENTATION

The implementation of the Security Monitoring System on AWS was designed to demonstrate the practical integration of key AWS services for real-time security event detection and automated alerting. The primary services utilized in this implementation include AWS CloudTrail, Amazon CloudWatch, Amazon SNS, AWS Secrets Manager, IAM, and Amazon S3. The following steps outline the structured development and configuration of the monitoring system.

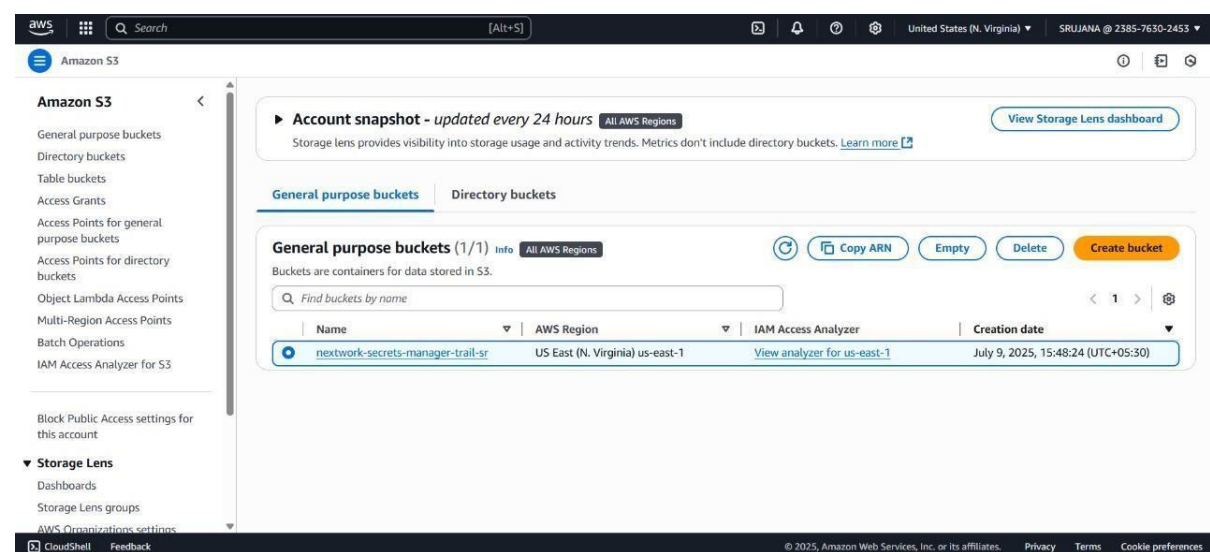
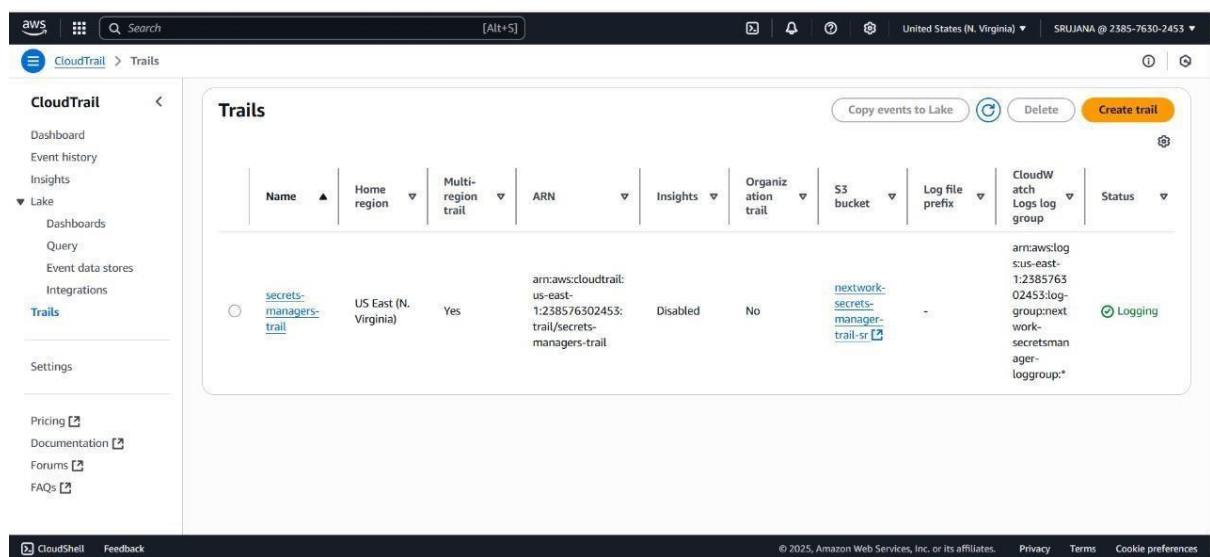
4.1 Secret Creation Using AWS Secrets Manager:

The initial step involved configuring AWS Secrets Manager to securely store sensitive data. A secret titled "TopSecretInfo" was created, representing confidential information typically subject to access restrictions. This setup ensured the inclusion of access-controlled data, enabling validation of monitoring mechanisms. Secrets Manager also offered built-in encryption, fine-grained access policies, and integration with CloudTrail for access tracking.



4.2 Configuration of AWS CloudTrail:

AWS CloudTrail was enabled to record all API activities related to the AWS account. CloudTrail was configured to monitor management events, specifically GetSecretValue operations associated with Secrets Manager. Both read and write API events were selected to ensure complete visibility, although the monitoring was primarily focused on write operations, which include access to secrets due to their sensitivity. The captured events were stored in Amazon S3, enabling durable and auditable log storage



Event name	Event time	User name	Event source	Resource type	Resource name
ListSecrets	July 10, 2025, 07:07:04 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	-	-
ListSecrets	July 10, 2025, 07:07:04 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	-	-
GetSecretValue	July 09, 2025, 23:13:01 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	AWS::SecretsManager::...	arn:aws:secretsmanag...
GetResourcePolicy	July 09, 2025, 23:07:40 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	AWS::SecretsManager::...	arn:aws:secretsmanag...
DescribeSecret	July 09, 2025, 23:07:39 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	AWS::SecretsManager::...	TopSecretInfo
ListSecrets	July 09, 2025, 23:07:24 (UTC+0...)	SRUJANA	secretsmanager.amazo naws.com	-	-

4.3 Verification of CloudTrail Event Capture:

To validate the logging capability, the secret was retrieved through two methods: Via the Secrets Manager Console using the "Retrieve secret value" option. Through the AWS CLI, using the get-secret-value command within CloudShell. Both actions secrets successfully triggered GetSecretValue events, which were recorded in CloudTrail and made visible in the Event History. This confirmed CloudTrail’s ability to track sensitive API actions accurately.

TopSecretInfo

Secret details

Encryption key
aws/secretsmanager

Secret name

Secret description

Secret created for security monitoring on aws

CloudShell

```

$ aws secretsmanager get-secret-value --secret-id TopSecretInfo --region us-east-1
{
  "ARN": "arn:aws:secretsmanager:us-east-1:238576302453:secret:TopSecretInfo-f06e0",
  "Name": "TopSecretInfo",
  "VersionId": "fbac0e7-dddc-4b25-b478-14fc3d8024a",
  "SecretString": "{\"the API key is\":\"build a security monitoring on aws123\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2025-07-09T09:05:11.640000+00:00"
}
```

4.4 CloudWatch Metrics Configuration:

Amazon CloudWatch was integrated to analyze CloudTrail logs and generate metric-based insights. A metric filter was configured to detect log entries corresponding to secret access. Each matching event incremented the metric value by one, enabling the creation of usage patterns. The metric configuration also included a default value for periods when the defined event did not occur, ensuring consistent metric evaluation.

Assign metric

Create filter name

Log events that match the pattern you define are recorded to the metric that you specify. You can graph the metric and set alarms to notify you.

Filter name

GetSecretsValue

Filter pattern

"GetSecretValue"

☐

Enable metric filter on transformed logs

When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Metric details

Metric namespace

Namespaces let you group similar metrics. [Learn more](#)

SecurityMetrics

Create new

Namespaces can be up to 255 characters long; all characters are valid except for colon(:) at the start of the name.

Metric name

Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)

Secret is accessed

Metric name can be up to 255 characters long; all characters are valid except for colon(:), asterisk(*), dollar(\$), and space().

Metric value

Metric value is the value published to the metric name when a Filter Pattern match occurs.

1

Valid metric values are: floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$.status for JSON-based filter pattern - dollar (\$) or dollar dot (\$) followed by alphanumeric and/or underscore (_) characters).

Default value - optional

The default value is published to the metric when the pattern does not match. If you leave this blank, no value is published when there is no match. [Learn more](#)

0

Unit - optional

Select a unit

4.5 Setting Up CloudWatch Alarms:

A CloudWatch Alarm was created to monitor the custom metric. The alarm was set to trigger if the number of secret access events exceeded a threshold—specifically, if the average number of GetSecretValue API calls was greater than one within a five-minute period. This setup enabled real-time detection of unauthorized or excessive access to sensitive data



Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

`arn:aws:sns:us-east-1:238576302453:SecurityAlarms:d13a3ac3-3cbc-40de-86b7-7410ff4b95ef`

If it was not your intention to subscribe, [click here to unsubscribe](#).

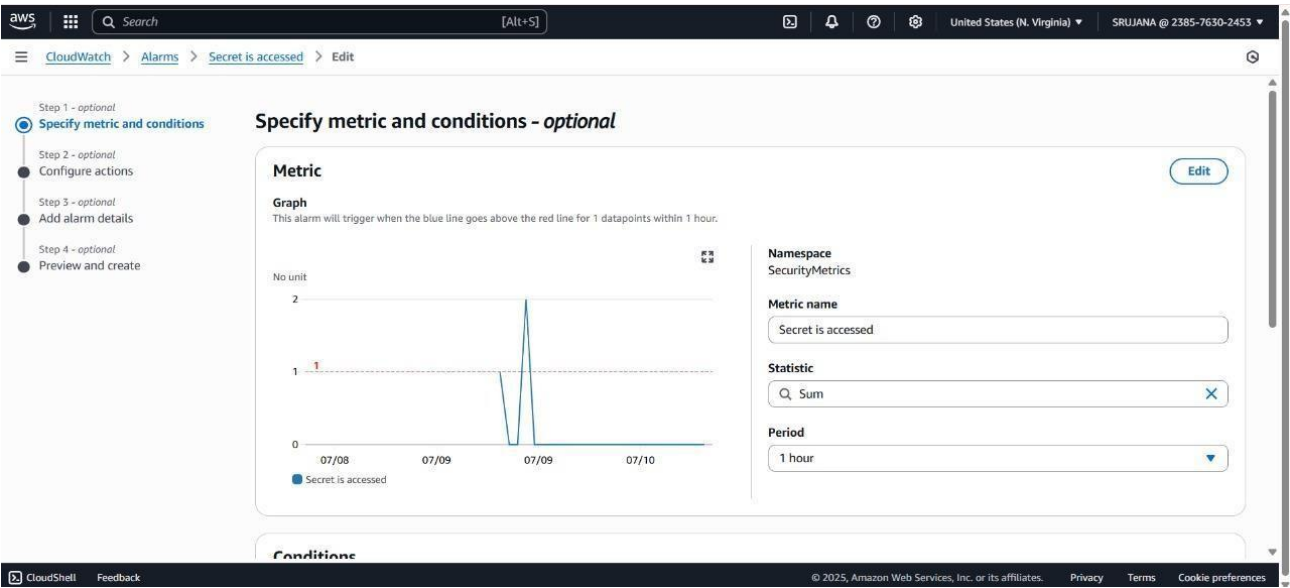
4.6 Integration with Amazon SNS for Notifications:

An Amazon SNS (Simple Notification Service) topic was established to send notifications when the CloudWatch Alarm was triggered. The topic was configured to dispatch email alerts to subscribed recipients, ensuring that designated stakeholders were promptly informed of critical security events. To prevent unsolicited messaging, SNS required email confirmation from all intended recipients prior to alert activation.

4.7 Troubleshooting and Configuration Adjustment:

During the validation phase of the security monitoring system, a critical issue was identified: email notifications were not being received, even though the configured secret in AWS Secrets Manager had been accessed multiple times. This discrepancy indicated a failure within the monitoring pipeline, necessitating a structured and thorough troubleshooting process to isolate and resolve the problem.

A systematic investigation was conducted to identify the root cause of failed email notifications despite secret access events being logged. Verification steps confirmed that GetSecretValue API calls were accurately recorded in CloudTrail and forwarded to CloudWatch Logs. The metric filter was correctly matching the relevant log entries; however, the CloudWatch Alarm was misconfigured to evaluate using the "Average" statistic instead of "Sum." This suppressed the alarm trigger despite valid events. After correcting the aggregation method to "Sum," the system was re-tested and performed successfully—logging the event, incrementing the metric, triggering the alarm, and delivering an email alert via SNS. This resolution highlighted the critical role of accurate metric evaluation and demonstrated the interconnected nature of AWS monitoring components in ensuring effective security alerting.



5. RESULTS

The implementation of the AWS Security Monitoring System led to successful configuration and validation of key AWS services for real-time event monitoring and alerting. The following outcomes demonstrate the effectiveness of the designed architecture and operational flow.

1. Secure Storage and Controlled Access of Secrets:

A key achievement was the use of AWS Secrets Manager to securely store sensitive information. Proper configuration ensured controlled access through IAM roles and policies. The system successfully tracked each access instance using CloudTrail, supporting secure and auditable operations.

2. Comprehensive Event Logging with AWS CloudTrail:

CloudTrail was configured to monitor management-level API events, including secret access activities. Events such as GetSecretValue were logged and verified via the CloudTrail console and AWS CLI. The system captured both read and write operations, validating CloudTrail's effectiveness in event tracking.

3. Real-Time Monitoring and Alerting via CloudWatch:

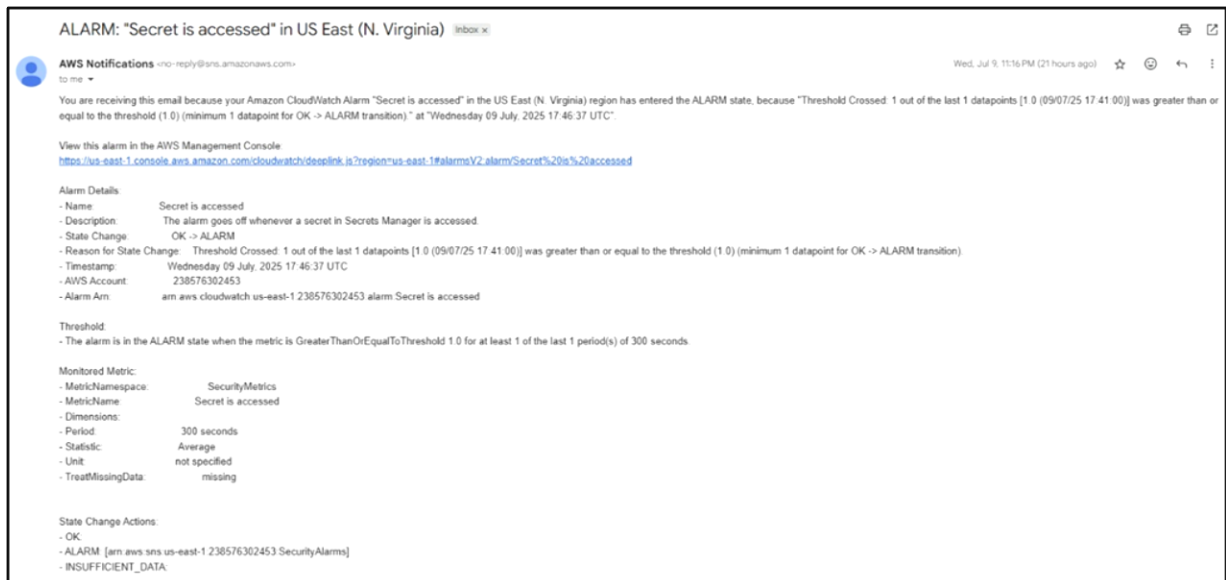
AWS CloudWatch played a central role in real-time analysis and alerting. A custom metric filter was established to monitor secret access logs. A CloudWatch Alarm was successfully triggered based on predefined thresholds, demonstrating real-time incident detection.

4. Notification Integration through SNS:

The system integrated Amazon SNS for notification delivery. Email alerts were automatically dispatched upon secret access, after verifying email subscriptions. This enhanced situational awareness and enabled immediate administrative response.

5.5.Effective Troubleshooting and System Validation:

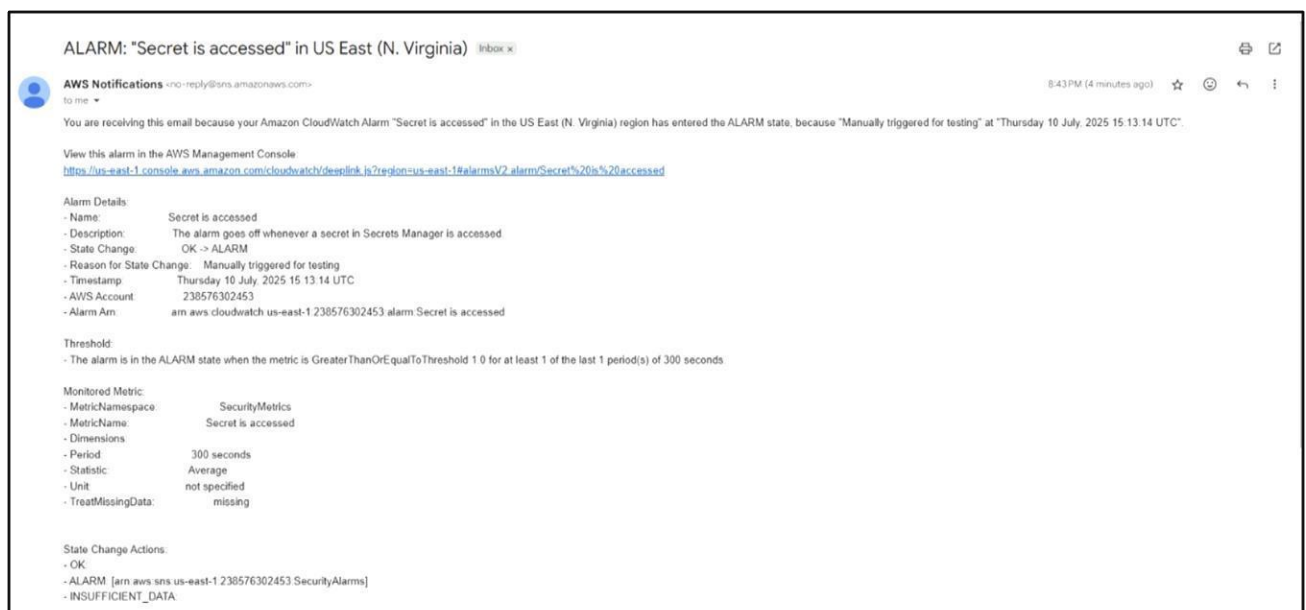
Initial issues with notification delivery were resolved by adjusting the alarm evaluation criteria from AVERAGE to SUM. Post-adjustment testing confirmed the full end-to-end functionality: secret access → log capture → metric match → alarm trigger → SNS notification.



Screenshots of the outputs:

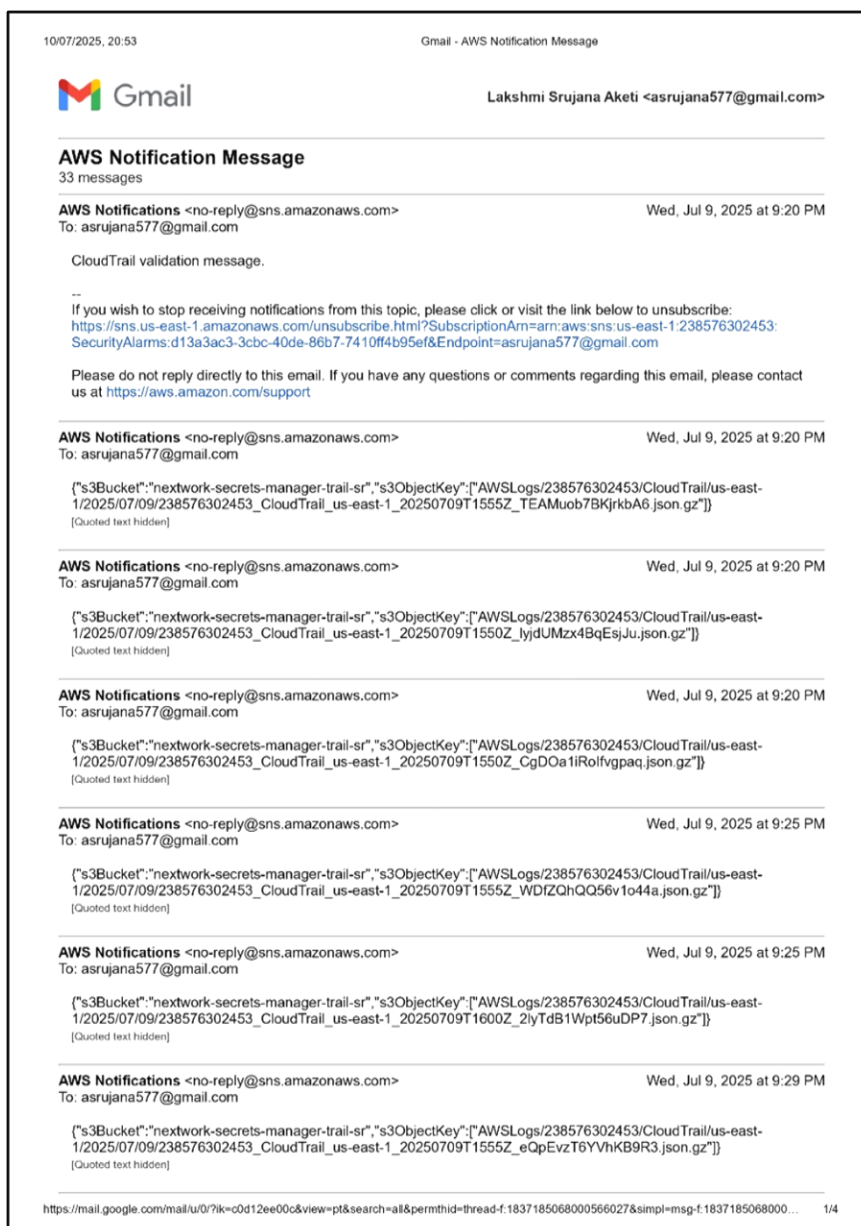
Success!

To validate that our monitoring system can successfully detect and alert when my secret is accessed, we checked my secret's value one more time. We received an email within 1-2 minutes of the event! Our alarm in CloudWatch is also in "In alarm" state.



5.6.Comparison of CloudTrail vs CloudWatch Notifications:

In a project extension, we enabled SNS notification delivery in CloudTrail because this lets us evaluate CloudTrail vs CloudWatch for notifying us about events like our secret getting access. After enabling CloudTrail SNS notifications, our inbox was very quickly filled with new emails from SNS (as it was notified by CloudTrail). In terms of the usefulness of these emails, we thought that we're receiving LOTS (it's a little overwhelming)and the logs themselves don't show what happened in terms of management events that occurred. We only see that new logs have been stored in our bucket.



CONCLUSION

The "Build a Security Monitoring System" project provided a hands-on opportunity to explore key AWS services and build a real-time alert system for monitoring sensitive data access. The goal was to detect when a secret stored in AWS Secrets Manager was accessed and to send immediate email notifications using Amazon SNS. This exercise helped us understand how to connect multiple AWS services to form a cohesive and secure monitoring pipeline. We started by creating a secret and configuring CloudTrail to log management events, specifically the GetSecretValue API call. These logs were then streamed into CloudWatch Logs, where we defined metric filters to detect when the secret was accessed. A CloudWatch metric was created to count the number of such events, and a CloudWatch alarm was set to trigger if more than one access occurred within a five-minute window. To notify us, we set up an SNS topic with an email subscription. Initially, we encountered a problem where alerts were not being delivered. Through careful troubleshooting, we discovered the alarm used the average instead of the sum statistic, which meant it never crossed the threshold to trigger. After correcting this, the system successfully sent real-time alerts when the secret was accessed. We also tested CloudTrail SNS notifications directly and found that while they were fast, they produced a high volume of less-informative messages. In contrast, the CloudWatch-based approach provided greater control and clarity. Overall, this project reinforced important cloud security principles, including centralized logging, metric-based alerting, and the value of monitoring sensitive resources. It was a valuable learning experience in securing cloud infrastructure and responding to security events effectively.